Aligning the TORA L2 Live Feed with the Research Tick Tape

Tick-level reconstruction, level/book validation strategy, and the 98.4% dry-run gate

obgx ↔ openctp-forked live-service iteration
2026-08-27

Current stage (2026-08-27).

- Live-side chain *built and verified offline* on real past data aligned with place/trade: research tape re-dressed as a TORA capture reproduces the research tick_table at **98.4% cell-exact** (21,719 rows), event codes / sign / volumes each **99.9%** (§5); the capture→convert round-trip on the tape itself is bit-faithful (100.0%).
- Row-generation contract extracted from cn_rst3 and pinned (`alignment_ob/tick_row_spec.md`); normalizer, converters, profiler, and the dry-run gate are all committed and tested (53 tests).
- TORA SDK v4.0.8 in hand; C++ capture tool built and linked; both fronts reachable from the working host; first login unanswered (pre-market / fund-account-login DepartmentID); **live capture scheduled for the 09:15 Beijing market open** (§8).

What to do next (in order).

1. Run the live capture at Beijing open; grade it with the exact dry-run machinery of §4; resolve login (UserID vs fund account + DepartmentID).
2. Port the book reconstruction from cn_rst3 driven by the TORA order-by-order primitives; validate against the vendor 10-level snapshots as *reference* (§6). Closes `level_placed` beyond the visible 10 levels and the book-stamp-timing deltas.
3. Implement the per-order row-mutation port (continuous fills mutate and re-emit the aggressor's Add row) — closes the remaining 7,749 tape rows (§7).
4. Offline acceptance for (2)+(3): place/trade → port → tick_table at **100.0%**.
5. Then: M3 same-date profile → M4 live cqr_bar → M5 execution.

Abstract

We are connecting the obgx research stack (place/trade → book reconstruction → `tick_table` → cqr bar graphs → cc models) to a live Chinese-equity feed — the TORA (HuaXin Qidian) native L2 API, which delivers order-by-order order and trade streams plus a 10-level book snapshot stream. The central question is the alignment between *our data* (the reconstructed research tick tape) and *their live data* (the vendor L2 streams). We extracted the exact row-generation contract from the Rust engine (cn_rst3), re-dressed the research tape as a TORA-shaped capture, and measured the live pipeline's fidelity on real 20220104 data: **98.4% cell-exact** across the statelessly-representable rows (21,719 rows), with event codes, signs and volumes each **99.9%** exact. The residual deltas are precisely attributed, and the strategy to close them — reconstruct the book from the vendor's own order-by-order updates and use the vendor's level snapshots only as a *reference* for the reconstruction — is specified here together with the remaining per-order row-mutation port.

1 The two data worlds

Research-side (our data):

- **place/ + trade/** raw parquet (the exchange feed in broker format: order adds/cancels, trades), consumed by cn_rst3;
- **book reconstruction** — the Rust engine maintains a full limit-order book with FIFO slots from place/trade and emits the **tick_table**: 83 columns (24 core + 60 book), one row per executable event, prices in integer cents, timestamps in ns past midnight;
- tick_table feeds the cqr bar graphs (per 3m window) and onward to the cc models. Bar fidelity to this tape is pinned bit-faithfully.

Live-side (their data): the TORA L2 API (`TORALEV2API` v4.0.8) delivers three streams per instrument:

- **md** — 10-level book snapshot (`Lev2MarketDataField`): prices (yuan), quantities, per-level order counts, session state (auction/continuous), cumulative volume;
- **od** — order-by-order order stream (`Lev2OrderDetailField`): add/delete of resting orders with order number, price, quantity, side, order status;
- **tr** — order-by-order trade stream (`Lev2TransactionField`): price, quantity, buy/sell order numbers, aggressor flag (BSFlag), sequence numbers; resend variants exist for gap recovery.

2 Format alignments that matter

Extracted from the Rust sources (`tick_row.rs`, `engine/tick.rs`) and confirmed against the 20220104 tape — these are the format-level contracts the live path must reproduce:

Property	Contract
Prices	integer cents in tick_table; double yuan on the wire ($\times 100$)
Volumes	shares, verbatim
Timestamps	TORA wire is packed `HHMMSSmmm`; the tape is 100% ms-aligned
`local_ts`	= `exch_ts` + 100 ms (fixed offset)
`order_seq`	per-symbol row counter (`row_index + 1`)
Trade file	*aggregated*: one auction print fans out to many tape rows
TORA `tr`	per-fill — exactly tape granularity
`vwap_better`	the single float column (all others int64)

3 Row-generation contract

Code	Meaning	`vol_placed`	`vol_traded`
1	Add (limit order to book)	$+q$	0
2	Cancel	$-q$	0
4	Auction add / auction fill	$+q$ / 0	0 / q
5	Market / aggressive-intent row	$+q$	0

Two structural facts drive the live design:

1. **Trades do not emit rows.** A continuous fill *mutates* the aggressor's existing Add row (`vol_traded`, `tvr`, `count_traded`, `depth_traded`) and the updated row is *re-emitted* (sweep-confirm re-emission). A stateless mapper cannot reproduce this; it requires the per-order row-mutation port.

2. `level_placed` is the number of distinct price levels between the best price and the order's price (0 at best, −1 crossed, −2 auction fills) computed on the *full* book — values ≥ 10 appear in the tape. A 10-level feed alone can only compute it while the order price is within the visible levels (the cqr graph's `_lle1` variants need only the top two levels and remain exact).

Book snapshots on a row are *pre-event* (the book as it stood before the row's own event). Auction windows (empirical, sh600000/20220104): adds 09:15–09:25 and 14:57–15:00; the auction fill print is a single instant (09:25:00.980 open — 67 fan-out rows at one price; 15:00:00.000 close).

4 Methodology: the re-dress dry run

Because the TORA wire and the research tape describe the same underlying market, the live pipeline can be validated *offline* on past data aligned with place/trade:

1. **Re-dress**: research place → TORA `od` lines; trade → `tr` lines; tick_table book columns → `md` lines — all in the exact wire format of the C++ capture tool (`lev2_dump.cpp`).
2. **Normalize** (`normalize.py`): merge the three streams in time order, maintain the last 10-level book, stamp every event pre-event, dedupe resends in the live sequence space, map codes per §3.
3. **Convert** (`converters.py`): events → the canonical 83-column tick_table schema.
4. **Profile** (`profiler.py`): cellwise comparison against the research tick_table keyed on (`exch_ts` ms, price, qty), with per-column mismatch attribution, NaN-exact semantics, and missing-row/column reporting.

The loop is pinned by `tests/test_synth_tora.py`; the identical chain grades any real capture from the live front.

5 Results

Baseline: the capture→event→convert round-trip on the research tape itself is **bit-faithful** — sh600000/20220104, 30,598 rows × 83 columns, **100.0%** exact (`tests/test_alignment.py`).

Dry run (real 20220104, adds/cancels/auction fills — the statelessly representable 73% of the tape):

Metric		Value
Rows compared		21,719
Overall cell-exact		**98.4%**
Event-code mismatches	25	(99.9%)
Sign mismatches	15	(99.9%)
`vol_placed`/`vol_traded` mismatches	135 / 122	(99.9%)
`level_placed` mismatches		5,373
Book qty/order-count mismatches		∼1,100

Residual attribution:

- **`level_placed` beyond the visible 10 levels** (5,373 rows): a structural limit of any 10-level feed; closed by the reconstruction strategy of §6.

- **Book-stamp timing at fan-out instants** (∼1,100): at multi-fill instants the pre-event book differs by the fill sequence; closed by reconstructing the book from the tick stream itself rather than from snapshots.
- **Continuous trades** (7,749 rows): mutation-only rows; require the per-order row-mutation port (§7).

6 Level and book strategy: reconstruct from ticks, use their levels as reference

The vendor sends both the order-by-order event stream *and* its own book snapshots. The proposed strategy is to invert their roles relative to a snapshot-trusting design:

1. **Primary book = our reconstruction from their ticks.** Port the cn_rst3 book machinery (limit-order book with FIFO slots, cages, sweeps) driven by the TORA `od/tr` primitives — the same add/cancel/fill events the research reconstruction consumes. This yields the full-depth book, exact `level_placed`, exact pre-event stamping, and the mutation/re-emission semantics (§7) — i.e., the whole 83-column tape.
2. **Vendor levels = reference, not source.** Their 10-level snapshots validate the reconstruction: per-level cross-checks at snapshot instants, divergence alerts, and drift statistics. The vendor book is never used as the primary book state.
3. **Why this order.** (a) The research pipeline itself reconstructs — parity requires reconstructing; (b) the vendor snapshot timing relative to their own events is unknown until M0 captures (the dry run's book-stamp deltas are exactly this uncertainty); (c) beyond-10 information only exists in the reconstruction.

7 The remaining port

The per-order row-mutation port is the one substantive implementation left: an OrderBook with FIFO slots and cages, per-order row state mutated by fills, sweep handling (SZ), and re-emission — verified offline against place/trade → tick_table at 100.0% (the same gate `scripts/gate_stream_vs_batch.py` applies to the Rust engine). The TORA feed supplies exactly the primitives the port consumes, so the live engine then reproduces the full research semantics on a live feed.

8 Live capture status

The official SDK (`API_Stock_lv2_C++ v4.0.8`) is in hand; the C++ capture tool is built and linked; both fronts (7×24 and simulation) are reachable from the working host. First login at 06:15 Beijing returned no response (pre-market and/or the account-login DepartmentID requirement); a capture is scheduled for the 09:15 Beijing market open. Every capture is graded by the exact machinery of §4.

9 Conclusion

The alignment between our research tape and the TORA live feed is now a measured, attributed quantity rather than an assumption: 98.4% cell-exact with 99.9% on the semantic core, and a precise closing plan — tick-driven book reconstruction validated against vendor levels, plus the per-order mutation port — that targets full parity on the live feed.